

FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

Filière Ingénierie des Systèmes Intelligents (ISI)

Mini-Projets Programmation En Python

Semestre 1 : 2024/2025

Réalisé par :

El Akkaoui Adam & Merizak Mehdi

Intitulé

**Développement d'un mini-projet de gestion en
Python en deux versions de stockage (Mini projet 5)**

Résumé

Le présent projet, intitulé "**Développement d'une application de gestion en Python avec deux modes de stockage**", a été réalisé dans le cadre d'un exercice académique visant à renforcer nos compétences en programmation et en gestion des données.

L'objectif principal est de concevoir et développer une application permettant la gestion de plusieurs entités (produits, clients, commandes et fournisseurs) en mettant en œuvre les opérations **CRUD** (Create, Read, Update, Delete). Deux approches de stockage ont été explorées : l'une basée sur un **fichier texte**, et l'autre sur une **base de données SQLite3**.

Ce projet nous a permis d'approfondir notre compréhension de la **programmation orientée objet (POO)** en Python, tout en appliquant des principes de gestion et de structuration des données. Il met en évidence les avantages et les limites des différentes méthodes de stockage et illustre leur impact sur l'efficacité et la maintenance du système.

Le rapport détaille la conception du projet, les choix techniques adoptés, ainsi que les tests réalisés. Il propose également une analyse des contraintes rencontrées et des perspectives d'amélioration pour rendre l'application plus performante et évolutive.

Table des matières

I.	Contexte et Objectif du Projet.....	6
I.1	Contexte	6
I.2	Objectif.....	6
I.3	Conclusion	7
II.	Conception détaillée du projet avec UML	8
II.1	Introduction	8
II.2	Outil de modélisation	8
II.3	Diagramme de classe	9
II.4	Diagramme de cas d'utilisation.....	9
II.5	Conclusion	10
III.	Étapes de développement.....	11
III.1	Analyse des besoins.....	11
III.2	Environnement de développement.....	11
III.3	Conception et architecture.....	11
III.4	Modèle Objet et Structure des Classes	12
III.4.1	Classe Client.....	12
III.4.2	Classe Produit	14
III.4.3	Classe Commande	15
III.4.4	Classe Fournisseur	17
III.4.5	L'utilisation des méthodes statiques	17
III.5	Différences entre SQLite et Fichiers Texte dans l'implémentation des méthodes.....	17
III.6	Menu Utilisateur et Navigation.....	17
III.6.1	Structure du Menu et Navigation	17
III.6.2	Différences entre SQLite et Fichier Texte dans le menu	18
IV.	Tests réalisés	19
IV.1	Conclusion	21
V.	Limites et Perspectives d'Amélioration	23
V.1	Limites du projet	23
V.2	Perspectives d'amélioration.....	23
V.3	Conclusion	24

Introduction Générale

Dans un monde où l'informatisation des processus est devenue essentielle, la programmation joue un rôle clé dans la gestion des informations. Parmi les langages les plus populaires, **Python** s'impose comme un choix privilégié grâce à sa simplicité, sa lisibilité et sa large gamme de bibliothèques adaptées à divers domaines, notamment la gestion des données et l'automatisation des tâches.

Dans le domaine de la gestion, les entreprises doivent manipuler de nombreuses informations et un bon système de gestion permet non seulement d'optimiser l'organisation des données, mais aussi d'améliorer l'efficacité et la prise de décision. Python, avec ses capacités de manipulation de fichiers, ses bases de données intégrées et sa flexibilité, constitue un outil puissant pour développer des applications de gestion adaptées aux besoins spécifiques.

Ce projet s'inscrit dans cette logique d'apprentissage et d'application concrète des concepts de **programmation orientée objet (POO)** et de **gestion des données** en Python. Il vise à concevoir une application capable de gérer des entités telles que des produits, des clients, des commandes et des fournisseurs en utilisant deux modes de stockage : un fichier texte et une base de données SQLite3.

Au-delà de la mise en œuvre des opérations fondamentales de gestion (ajout, suppression, modification et consultation des données), ce projet permet d'acquérir une meilleure compréhension des avantages et des limites de différentes méthodes de stockage, tout en appliquant des bonnes pratiques de développement logiciel.

I. Contexte et Objectif du Projet

I.1 Contexte

Dans le cadre de l'apprentissage du langage Python et de ses capacités en gestion de données, ce projet a été réalisé pour mettre en pratique la programmation orientée objet et les opérations de manipulation de données à travers deux types de stockage : un fichier texte (.txt) et une base de données SQLite3.

Dans de nombreuses entreprises, la gestion des clients, des produits, des commandes et des fournisseurs est essentielle pour assurer le bon fonctionnement des opérations commerciales. Les informations relatives à ces éléments doivent être correctement stockées, accessibles et manipulées afin d'optimiser les flux de travail. Cependant, la gestion manuelle de ces données peut entraîner des erreurs et des pertes de temps.

Ce projet vise à simuler un tel système en offrant une solution simple et fonctionnelle capable de stocker, récupérer, mettre à jour et supprimer des données de manière efficace.

I.2 Objectif

L'objectif principal du projet est de développer une application en Python capable d'effectuer les opérations CRUD (Create, Read, Update, Delete) sur un ensemble d'entités préalablement définies. Le projet respecte certaines contraintes, notamment :

- Utiliser un maximum de **quatre classes/entités** représentant des éléments de gestion courants (**Client, Commande, Produit, Fournisseur**).
- Implémenter deux modes de stockage des données :
 1. **Version 1 : Stockage en fichier texte (.txt)** – Les informations sont sauvegardées sous forme de lignes dans un fichier texte.
 2. **Version 2 : Stockage en base de données SQLite3** – Utilisation d'une base de données relationnelle avec des tables structurées.

I.3 Conclusion

Ce projet vise à renforcer la compréhension des concepts fondamentaux de la programmation et à acquérir une première expérience dans le développement de logiciels de gestion simples mais fonctionnels en langage python.

II. Conception détaillée du projet avec UML

II.1 Introduction



UML, c'est l'acronyme anglais pour « Unified Modeling Language ». On le traduit par « Langage de modélisation unifié ». La notation UML est un langage visuel constitué d'un ensemble de schémas, appelés des diagrammes, qui donnent chacun une vision différente du projet à traiter. UML nous fournit donc des diagrammes pour représenter le logiciel à développer, son fonctionnement, sa mise en route, les actions susceptibles d'être effectuées par le logiciel, etc.

Nous avons choisi de travailler avec UML parce qu'il exprime mieux la vue statique et dynamique du système d'information et, pour notre application Web, il est nécessaire de faire une analyse très approfondie pour pouvoir dégager les nécessités de développement ainsi que quelques scénarios d'exécution.

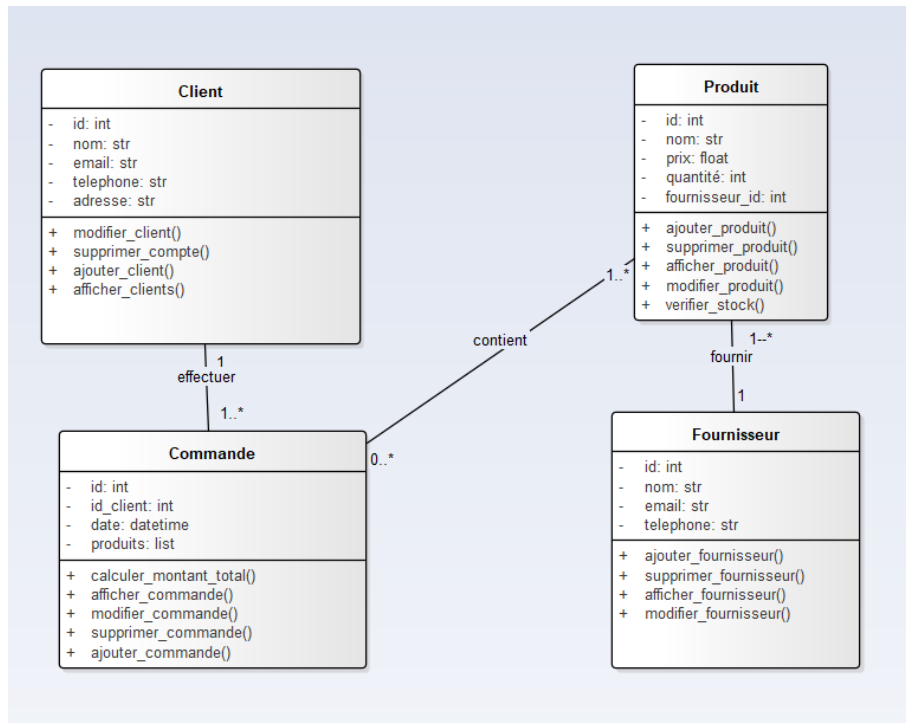
II.2 Outil de modélisation



Enterprise Architect est un logiciel de modélisation et de conception UML, édité par la société australienne Sparx Systems. Couvrant, par ses fonctionnalités, l'ensemble des étapes du cycle de conception d'application, il est l'un des logiciels de conception et de modélisation les plus reconnus.

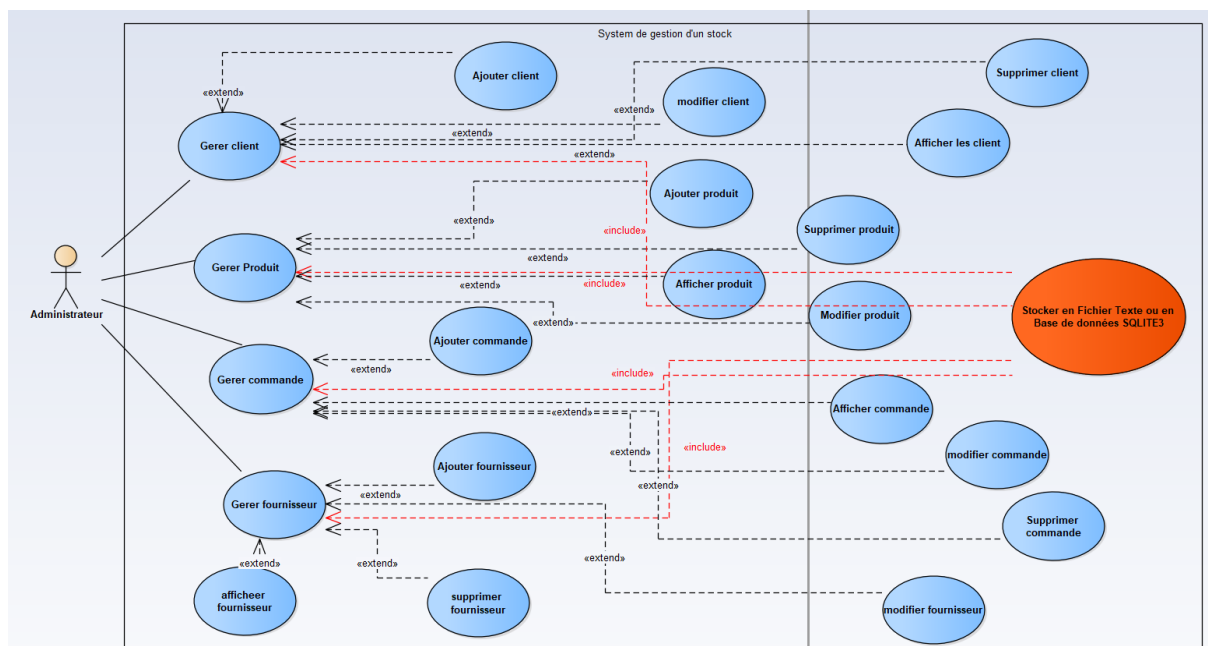
II.3 Diagramme de classe

Le diagramme de classes illustre les classes principales du système et leurs relations.



II.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation illustre les interactions entre l'administrateur et le système de gestion d'un stock.



II.5 Conclusion

Le deuxième chapitre a été consacré à la conception du système de gestion d'un stock. Nous avons élaboré les différents modèles de données et de traitement nécessaires pour assurer le bon fonctionnement du système. La phase de conception a permis de structurer et de définir les interactions entre les différentes composantes du projet, en tenant compte des besoins des utilisateurs et des exigences techniques. En conclusion, ce chapitre a permis de créer une architecture claire et cohérente, prête pour l'implémentation. La conception détaillée du projet constitue une étape cruciale, garantissant que toutes les fonctionnalités requises sont correctement intégrées et prêtes à être développées.

III. Étapes de développement

III.1 Analyse des besoins

Les deux programmes de gestion de stock ont été développés avec des approches différentes en matière de stockage des données :

- **Gestion_Stock_Sql.py** utilise **SQLite** pour stocker les données dans une base relationnelle.
- **Gestion_Stock_text.py** repose sur des fichiers **Texte** pour stocker les informations sous format texte.

Ces choix influencent la manière dont les données sont structurées, manipulées et sécurisées, comme détaillé ci-dessous.

III.2 Environnement de développement



Le projet a été développé en Python en utilisant l'IDE **PyCharm**, choisi pour ses outils avancés de gestion de projet, son débogueur intégré et sa compatibilité avec les bases de données et JSON. PyCharm a facilité l'organisation du code, l'exécution des tests et le suivi des erreurs, garantissant ainsi un développement fluide et structuré.

III.3 Conception et architecture

Les deux programmes suivent une architecture orientée objet, avec des classes représentant les entités suivantes :

- **Client** : gestion des clients.
- **Produit** : gestion des produits en stock.
- **Commande** : gestion des commandes et des stocks associés.
- **Fournisseur** : gestion des fournisseurs.

Différences clés

Composant	Gestion_Stock_Sql.py (Base de données)	Gestion_Stock_text.py (Fichiers Texte)
Bibliothèques utilisées	sqlite3, datetime	Json,os,datetime
Stockage	SQLite (base relationnelle)	Fichiers Texte
Requêtes	SQL (SELECT, INSERT, UPDATE, DELETE)	Manipulation de fichiers Texte (load_data (), save_data ())
Gestion de l'intégrité des données	Contraintes SQL (FOREIGN KEY, PRIMARY KEY)	Vérifications manuelles dans le code
Performances	Optimisé pour les grandes quantités de données	Moins performant, dépend des fichiers texte
Persistance des données	Stockage structuré et sécurisé	Données stockées en texte, plus vulnérables

III.4 Modèle Objet et Structure des Classes

III.4.1 Classe Client

- **Attributs** : id, nom, email, téléphone, adresse

```
class Client: 7 usages
    def __init__(self, nom, email, telephone, adresse, id=None):
        self.id = id or self._generate_id()
        self.nom = nom
        self.email = email
        self.telephone = telephone
        self.adresse = adresse
```

- **Méthodes principales** :
 - **Ajouter_client ()** : Ajoute un client à la base de données SQL ou au fichier texte

```
def ajouter_client(self): 1 usage
    clients = load_data(CLIENTS_FILE)
    clients.append(self.to_dict())
    save_data(CLIENTS_FILE, clients)
```

```
def ajouter_client(self): 1 usage
    cursor.execute( sql: '''INSERT INTO clients (nom, email, telephone, adresse)
                    VALUES (?, ?, ?, ?)''', parameters: (self.nom, self.email, self.telephone, self.adresse))
    conn.commit()
```

- **Afficher_clients ()** : Affiche la liste des clients

```
@staticmethod 4 usages
def afficher_clients():
    cursor.execute("SELECT * FROM clients")
    clients = cursor.fetchall()
    print("\nListe des clients :")
    for client in clients:
        print(f"ID : {client[0]}, Nom : {client[1]}, Email : {client[2]}, Téléphone : {client[3]}, Adresse : {client[4]}")
```

```
@staticmethod 4 usages
def afficher_clients():
    clients = load_data(CLIENTS_FILE)
    print("\nListe des clients :")
    for client in clients:
        print(f"ID : {client['id']}, Nom : {client['nom']}, Email : {client['email']}, "
              f"    Téléphone : {client['telephone']}, Adresse : {client['adresse']}")
```

- **Modifier_client (client_id, nom, email, téléphone, adresse)** :
Modifie les informations d'un client

```
@staticmethod 1 usage
def modifier_client(client_id, nom, email, telephone, adresse):
    cursor.execute( sql: '''UPDATE clients SET nom = ?, email = ?, telephone = ?, adresse = ? WHERE id = ?''',
                    parameters: (nom, email, telephone, adresse, client_id))
    conn.commit()
```

```
@staticmethod 1 usage
def modifier_client(client_id, nom, email, telephone, adresse):
    clients = load_data(CLIENTS_FILE)
    for client in clients:
        if client['id'] == client_id:
            client.update({
                'nom': nom,
                'email': email,
                'telephone': telephone,
                'adresse': adresse
            })
            break
    save_data(CLIENTS_FILE, clients)
```

- **Supprimer_compte(client_id)** : Supprime un client

```
@staticmethod 1 usage
def supprimer_compte(client_id):
    clients = load_data(CLIENTS_FILE)
    clients = [c for c in clients if c['id'] != client_id]
    save_data(CLIENTS_FILE, clients)
```

```
@staticmethod 1 usage
def supprimer_compte(client_id):
    cursor.execute( sql: "DELETE FROM clients WHERE id = ?", parameters: (client_id,))
    conn.commit()
```

💡 **Remarque** : Ces méthodes suivent la même structure que celles des autres classes (Produit, Commande, Fournisseur).

III.4.2 Classe Produit

Cette classe fonctionne de manière similaire à Client, avec des méthodes identiques adaptées aux produits.

Différences principales :

- **Verifier_stock(produit_id)** : permet de vérifier la quantité disponible avant de valider une commande.

```
@staticmethod 2 usages
def verifier_stock(produit_id):
    produits = load_data(PRODUITS_FILE)
    for produit in produits:
        if produit['id'] == produit_id:
            return produit['quantite']
    return 0
```

```
@staticmethod 2 usages
def verifier_stock(produit_id):
    cursor.execute( sql: "SELECT quantite FROM produits WHERE id = ?", parameters: (produit_id,))
    quantite = cursor.fetchone()
    return quantite[0] if quantite else 0
```

- **Ajouter_produit ()** : met à jour la quantité et lie chaque produit à un fournisseur.

```
def ajouter_produit(self): 1 usage
    cursor.execute(sql: '''INSERT INTO produits (nom, prix, quantite, fournisseur_id)
                        VALUES (?, ?, ?, ?)''', parameters: (self.nom, self.prix, self.quantite, self.fournisseur_id))
    conn.commit()
```

```
def ajouter_produit(self): 1 usage
    produits = load_data(PRODUITS_FILE)
    produits.append(self.to_dict())
    save_data(PRODUITS_FILE, produits)
```

III.4.3 Classe Commande

Cette classe reprend la logique des autres, mais avec une particularité :

- **Ajouter_commande ()** : effectue une vérification du stock avant d'enregistrer la commande et met à jours le stock.

```
def ajouter_commande(self): 1 usage
    # Vérifier le stock pour chaque produit
    for produit in self.produits:
        stock_actuel = Produit.verifier_stock(produit['produit_id'])
        if stock_actuel < produit['quantite']:
            print(f"Stock insuffisant pour le produit {produit['produit_id']}. "
                  f"Commande non ajoutée pour ce produit.")
            return False

    # Mettre à jour les stocks et sauvegarder la commande
    commandes = load_data(COMMANDES_FILE)
    commandes.append(self.to_dict())
    save_data(COMMANDES_FILE, commandes)

    # Mettre à jour les stocks
    produits = load_data(PRODUITS_FILE)
    for commande_produit in self.produits:
        for produit in produits:
            if produit['id'] == commande_produit['produit_id']:
                produit['quantite'] -= commande_produit['quantite']
                break
    save_data(PRODUITS_FILE, produits)
    return True
```

```

def ajouter_commande(self): 1 usage
    cursor.execute( sql: '''INSERT INTO commandes (id_client, date) VALUES (?, ?)''', parameters: (self.id_client, self.date))
    conn.commit()
    self.id = cursor.lastrowid # Récupérer l'ID de la commande après insertion

    for produit_id, quantite in self.produits:
        # Vérifier si le stock est suffisant
        stock_actuel = Produit.verifier_stock(produit_id)
        if stock_actuel >= quantite:
            cursor.execute( sql: '''INSERT INTO produits_commandes (commande_id, produit_id, quantite)
                                VALUES (?, ?, ?)''', parameters: (self.id, produit_id, quantite))
            cursor.execute( sql: '''UPDATE produits SET quantite = quantite - ? WHERE id = ?''', parameters: (quantite, produit_id))
        else:
            print(f"Stock insuffisant pour le produit {produit_id}. Commande non ajoutée pour ce produit.")

    conn.commit()

```

- **Calculer_montant_total(commande_id)** : calcule le coût total d'une commande en fonction des produits commandés.

```

# Calculer le montant total et obtenir les détails des produits
montant_total = 0
produits_details = []
for commande_produit in commande['produits']:
    for produit in produits:
        if produit['id'] == commande_produit['produit_id']:
            montant_total += produit['prix'] * commande_produit['quantite']
            produits_details.append(
                f"{produit['nom']} (Quantité : {commande_produit['quantite']})"
            )
        break

```

```

@staticmethod 1 usage
def calculer_montant_total(commande_id):
    cursor.execute( sql: '''SELECT produits.prix, produits_commandes.quantite
                            FROM produits
                            JOIN produits_commandes ON produits.id = produits_commandes.produit_id
                            WHERE produits_commandes.commande_id = ?''', parameters: (commande_id,))
    items = cursor.fetchall()
    total = sum(prix * quantite for prix, quantite in items)
    return total

```

III.4.4 Classe Fournisseur

Fonctionne de manière similaire à **Client**, mais avec :

- Pas de gestion de stock.
- Un lien avec les produits (**fournisseur_id** dans **Produit**).

III.4.5 L'utilisation des méthodes statiques

Certaines méthodes, notamment **afficher_clients ()**, **afficher_produit ()**, **modifier_client ()**, et **supprimer_produit ()**, ont été définies comme **statiques** (**@staticmethod**). Cette approche a été choisie car ces méthodes ne nécessitent pas d'accéder aux attributs spécifiques d'une instance.

Elles effectuent uniquement des opérations sur la base de données (dans la version SQLite) ou sur les fichiers Texte (dans l'autre version). En les rendant statiques, nous évitons d'avoir à créer une instance de la classe pour chaque utilisation, ce qui optimise l'exécution et la gestion mémoire.

III.5 Différences entre SQLite et Fichiers Texte dans l'implémentation des méthodes

Fonctionnalité	SQLite (SQL)	Fichiers texte
Ajout d'un client	INSERT INTO clients VALUES (...)	Ajout d'un dictionnaire dans un fichier texte
Suppression d'un produit	DELETE FROM produits WHERE id=?	Filtrage et suppression dans la liste du fichier texte
Mise à jour d'une commande	UPDATE produits SET quantite= ? WHERE id= ?	Modification directe dans la structure du fichier texte
Vérification du stock	SELECT quantite FROM produits WHERE id=?	Parcours de la liste des produits dans le fichier texte

III.6 Menu Utilisateur et Navigation

III.6.1 Structure du Menu et Navigation

Le programme utilise une interface basée sur un **menu textuel** permettant à l'utilisateur de gérer les clients, produits, commandes et fournisseurs. Ce menu est

structuré de manière hiérarchique, avec un menu principal redirigeant vers des sous-menus dédiés à chaque entité du système.

Organisation du menu principal :

```
=== Menu Principal ===  
1. Gestion des clients  
2. Gestion des produits  
3. Gestion des commandes  
4. Gestion des fournisseurs  
5. Quitter  
Choisissez une option (1-5) :
```

Chaque option du menu dirige l'utilisateur vers un sous-menu spécifique. Par exemple, dans la gestion des produits, l'utilisateur peut ajouter, afficher, modifier ou supprimer un produit.

Exemple de sous-menu (Gestion des clients) :

```
=== Gestion des clients ===  
1. Ajouter un client  
2. Afficher les clients  
3. Supprimer un client  
4. Modifier un client  
5. Retour au menu principal  
Choisissez une option (1-5) :
```

Les choix sont gérés via des entrées utilisateur (`input()`), avec des conditions pour exécuter l'action correspondante.

III.6.2 Différences entre SQLite et Fichier Texte dans le menu

Bien que les menus des deux versions du programme soient identiques en apparence, il existe une différence majeure dans la gestion des données en arrière-plan :

- **Dans la version SQLite**, les opérations (ajouter, modifier, supprimer) sont exécutées via des **requêtes SQL** (INSERT, UPDATE, DELETE).

- Dans la version TEXTE, ces mêmes actions sont réalisées via la manipulation des fichiers texte (load_data (), save_data ()).

IV. Tests réalisés

Plusieurs tests ont été effectués pour valider le bon fonctionnement des deux programmes :

1. Tests unitaires sur les classes (Client, Produit, Commande, Fournisseur).
2. Tests de gestion des données :
 - Ajout de plusieurs clients, produits et commandes.
 - Vérification de l'affichage correct des données.

```
=== Gestion des clients ===
1. Ajouter un client
2. Afficher les clients
3. Supprimer un client
4. Modifier un client
5. Retour au menu principal
Choisissez une option (1-5) : 2

Liste des clients :
ID : 1, Nom : adam, Email : adam@gmail.com, Téléphone : 0612325487, Adresse : ouled oujih 612
ID : 2, Nom : mehdi, Email : mehdi@gmail.com, Téléphone : 0689215478, Adresse : le vallon 15
ID : 3, Nom : saad, Email : saad@gmail.com, Téléphone : 0714253669, Adresse : maghreb-arabi 71
ID : 4, Nom : reda, Email : reda@gmail.com, Téléphone : 0698163454, Adresse : Luchador 5
ID : 5, Nom : abderrahim, Email : chrerto@gmail.com, Téléphone : 0623987412, Adresse : maghreb-arabi 83
ID : 6, Nom : khalid, Email : khalid@gmail.com, Téléphone : 0721326549, Adresse : birrami 53
ID : 7, Nom : bilal, Email : bilal@gmail.com, Téléphone : 0612398247, Adresse : ouled oujih 19
ID : 8, Nom : mohammed, Email : monstis@gmail.com, Téléphone : 0698213265, Adresse : la ville-haute 14
ID : 9, Nom : alaa, Email : alaa@gmail.com, Téléphone : 0679351524, Adresse : la hay 96
ID : 10, Nom : sami, Email : sami@gmail.com, Téléphone : 0665063201, Adresse : centre-ville 203
ID : 11, Nom : yassine, Email : yassine@gmail.com, Téléphone : 0601200306, Adresse : haouzia 40
```

- Modification et suppression des entrées existantes.

```
=== Gestion des clients ===
1. Ajouter un client
2. Afficher les clients
3. Supprimer un client
4. Modifier un client
5. Retour au menu principal
Choisissez une option (1-5) : 3

Liste des clients :
ID : 1, Nom : adam, Email : adam@gmail.com, Téléphone : 0612325487, Adresse : ouled oujih 612
ID : 2, Nom : mehdi, Email : mehdi@gmail.com, Téléphone : 0689215478, Adresse : le vallon 15
ID : 3, Nom : salim, Email : salim@gmail.com, Téléphone : 0698765432, Adresse : La ville haute
ID : 4, Nom : Imad, Email : imad@gmail.com, Téléphone : 0654239856, Adresse : Essiyad 45
ID : 5, Nom : abderrahim, Email : chrerto@gmail.com, Téléphone : 0623987412, Adresse : maghreb-arabi 83
ID : 6, Nom : khalid, Email : khalid@gmail.com, Téléphone : 0721326549, Adresse : birrami 53
ID : 7, Nom : bilal, Email : bilal@gmail.com, Téléphone : 0612398247, Adresse : ouled oujih 19
ID : 8, Nom : mohammed, Email : monstis@gmail.com, Téléphone : 0698213265, Adresse : la ville-haute 14
ID : 9, Nom : alaa, Email : alaa@gmail.com, Téléphone : 0679351524, Adresse : la hay 96
ID : 10, Nom : sami, Email : sami@gmail.com, Téléphone : 0665063201, Adresse : centre-ville 203
ID : 11, Nom : yassine, Email : yassine@gmail.com, Téléphone : 0601200306, Adresse : haouzia 40
ID du client à supprimer : 11
```

```
=== Gestion des clients ===
1. Ajouter un client
2. Afficher les clients
3. Supprimer un client
4. Modifier un client
5. Retour au menu principal
Choisissez une option (1-5) : 4

Liste des clients :
ID : 1, Nom : adam, Email : adam@gmail.com, Téléphone : 0612325487, Adresse : ouled oujih 612
ID : 2, Nom : mehdi, Email : mehdi@gmail.com, Téléphone : 0689215478, Adresse : le vallon 15
ID : 3, Nom : salim, Email : salim@gmail.com, Téléphone : 0698765432, Adresse : La ville haute
ID : 4, Nom : reda, Email : reda@gmail.com, Téléphone : 0698163454, Adresse : Luchador 5
ID : 5, Nom : abderrahim, Email : chrerto@gmail.com, Téléphone : 0623987412, Adresse : maghreb-arabi 83
ID : 6, Nom : khalid, Email : khalid@gmail.com, Téléphone : 0721326549, Adresse : birrami 53
ID : 7, Nom : bilal, Email : bilal@gmail.com, Téléphone : 0612398247, Adresse : ouled oujih 19
ID : 8, Nom : mohammed, Email : monstis@gmail.com, Téléphone : 0698213265, Adresse : la ville-haute 14
ID : 9, Nom : alaa, Email : alaa@gmail.com, Téléphone : 0679351524, Adresse : la hay 96
ID : 10, Nom : sami, Email : sami@gmail.com, Téléphone : 0665063201, Adresse : centre-ville 203
ID : 11, Nom : yassine, Email : yassine@gmail.com, Téléphone : 0601200306, Adresse : haouzia 40
ID du client à modifier : 4
Nouveau nom : Imad
Nouveau email : imad@gmail.com
Nouveau téléphone : 0654239856
Nouvelle adresse : Essiyad 45
```

3. Tests de gestion du stock :

- Vérification que les quantités sont bien mises à jour après chaque commande.

```
=== Gestion des produits ===
1. Ajouter un produit
2. Afficher les produits
3. Supprimer un produit
4. Modifier un produit
5. Retour au menu principal
Choisissez une option (1-5) : 2

Liste des produits :
ID : 1, Nom : TV, Prix : 5000.0 DH, Quantité : 19, Fournisseur ID : 1
ID : 2, Nom : Recepteurs, Prix : 400.0 DH, Quantité : 25, Fournisseur ID : 1
ID : 3, Nom : Smartphones, Prix : 3500.0 DH, Quantité : 10, Fournisseur ID : 2
ID : 4, Nom : PC, Prix : 7500.0 DH, Quantité : 13, Fournisseur ID : 2
ID : 5, Nom : Playstation, Prix : 8000.0 DH, Quantité : 6, Fournisseur ID : 2
ID : 6, Nom : appareil photo, Prix : 1000.0 DH, Quantité : 9, Fournisseur ID : 2
ID : 7, Nom : machines à laver, Prix : 4499.0 DH, Quantité : 14, Fournisseur ID : 3
ID : 8, Nom : Réfrigérateur, Prix : 6000.0 DH, Quantité : 13, Fournisseur ID : 3
ID : 9, Nom : climatiseur, Prix : 5000.0 DH, Quantité : 40, Fournisseur ID : 4
ID : 10, Nom : chauffage, Prix : 1999.0 DH, Quantité : 32, Fournisseur ID : 5
```

```
Choisissez une option (1-5) : 1

Liste des clients :
ID : 1, Nom : adam, Email : adam@gmail.com, Téléphone : 0612325487, Adresse : ouled oujih 612
ID : 2, Nom : mehdi, Email : mehdi@gmail.com, Téléphone : 0689215478, Adresse : le vallon 15
ID : 3, Nom : salim, Email : salim@gmail.com, Téléphone : 0698765432, Adresse : La ville haute
ID : 4, Nom : Imad, Email : imad@gmail.com, Téléphone : 0654239856, Adresse : Essiyad 45
ID : 5, Nom : abderrahim, Email : chrerto@gmail.com, Téléphone : 0623987412, Adresse : maghreb-arabi 83
ID : 6, Nom : khalid, Email : khalid@gmail.com, Téléphone : 0721326549, Adresse : birrami 53
ID : 7, Nom : bilal, Email : bilal@gmail.com, Téléphone : 0612398247, Adresse : ouled oujih 19
ID : 8, Nom : mohammed, Email : monstis@gmail.com, Téléphone : 0698213265, Adresse : la ville-haute 14
ID : 9, Nom : alaa, Email : alaa@gmail.com, Téléphone : 0679351524, Adresse : la hay 96
ID : 10, Nom : sami, Email : sami@gmail.com, Téléphone : 0665063201, Adresse : centre-ville 203
ID du client : 1

Liste des produits :
ID : 1, Nom : TV, Prix : 5000.0 DH, Quantité : 19, Fournisseur ID : 1
ID : 2, Nom : Recepteurs, Prix : 400.0 DH, Quantité : 25, Fournisseur ID : 1
ID : 3, Nom : Smartphones, Prix : 3500.0 DH, Quantité : 10, Fournisseur ID : 2
ID : 4, Nom : PC, Prix : 7500.0 DH, Quantité : 13, Fournisseur ID : 2
ID : 5, Nom : Playstation, Prix : 8000.0 DH, Quantité : 6, Fournisseur ID : 2
ID : 6, Nom : appareil photo, Prix : 1000.0 DH, Quantité : 9, Fournisseur ID : 2
ID : 7, Nom : machines à laver, Prix : 4499.0 DH, Quantité : 14, Fournisseur ID : 3
ID : 8, Nom : Réfrigérateur, Prix : 6000.0 DH, Quantité : 13, Fournisseur ID : 3
ID : 9, Nom : climatiseur, Prix : 5000.0 DH, Quantité : 40, Fournisseur ID : 4
ID : 10, Nom : chauffage, Prix : 1999.0 DH, Quantité : 32, Fournisseur ID : 5
ID du produit à ajouter : 2
Quantité : 5
Ajouter un autre produit ? (oui/non) : non
```

```
=== Gestion des produits ===
1. Ajouter un produit
2. Afficher les produits
3. Supprimer un produit
4. Modifier un produit
5. Retour au menu principal
Choisissez une option (1-5) : 2

Liste des produits :
ID : 1, Nom : TV, Prix : 5000.0 DH, Quantité : 19, Fournisseur ID : 1
ID : 2, Nom : Recepteurs, Prix : 400.0 DH, Quantité : 20, Fournisseur ID : 1
ID : 3, Nom : Smartphones, Prix : 3500.0 DH, Quantité : 10, Fournisseur ID : 2
ID : 4, Nom : PC, Prix : 7500.0 DH, Quantité : 13, Fournisseur ID : 2
ID : 5, Nom : Playstation, Prix : 8000.0 DH, Quantité : 6, Fournisseur ID : 2
ID : 6, Nom : appareil photo, Prix : 1000.0 DH, Quantité : 9, Fournisseur ID : 2
ID : 7, Nom : machines à laver, Prix : 4499.0 DH, Quantité : 14, Fournisseur ID : 3
ID : 8, Nom : Réfrigérateur, Prix : 6000.0 DH, Quantité : 13, Fournisseur ID : 3
ID : 9, Nom : climatiseur, Prix : 5000.0 DH, Quantité : 40, Fournisseur ID : 4
ID : 10, Nom : chauffage, Prix : 1999.0 DH, Quantité : 32, Fournisseur ID : 5
```

- Vérification que les commandes ne dépassent pas les stocks disponibles

```
Liste des produits :
ID : 1, Nom : TV, Prix : 5000.0 DH, Quantité : 19, Fournisseur ID : 1
ID : 2, Nom : Recepteurs, Prix : 400.0 DH, Quantité : 20, Fournisseur ID : 1
ID : 3, Nom : Smartphones, Prix : 3500.0 DH, Quantité : 10, Fournisseur ID : 2
ID : 4, Nom : PC, Prix : 7500.0 DH, Quantité : 13, Fournisseur ID : 2
ID : 5, Nom : Playstation, Prix : 8000.0 DH, Quantité : 6, Fournisseur ID : 2
ID : 6, Nom : appareil photo, Prix : 1000.0 DH, Quantité : 9, Fournisseur ID : 2
ID : 7, Nom : machines à laver, Prix : 4499.0 DH, Quantité : 14, Fournisseur ID : 3
ID : 8, Nom : Réfrigérateur, Prix : 6000.0 DH, Quantité : 13, Fournisseur ID : 3
ID : 9, Nom : climatiseur, Prix : 5000.0 DH, Quantité : 40, Fournisseur ID : 4
ID : 10, Nom : chauffage, Prix : 1999.0 DH, Quantité : 32, Fournisseur ID : 5
ID du produit à ajouter : 1
Quantité : 20
Stock insuffisant pour ce produit.
Ajouter un autre produit ? (oui/non) :
```

4. Tests de persistance des données :

- Dans la version SQLite : vérification que les données restent accessibles après redémarrage du programme.
- Dans la version Texte : validation que les fichiers sont bien mis à jour après chaque opération.

```
{
  "id": 1,
  "id_client": 1,
  "date": "2025-02-03 14:09:16",
  "produits": [
    {
      "produit_id": 1,
      "quantite": 1
    },
    {
      "produit_id": 2,
      "quantite": 1
    }
  ]
},
```

IV.1 Conclusion

Le développement de cette application a suivi une approche méthodique, en respectant les principes de la programmation orientée objet et en assurant une séparation claire des responsabilités entre les différentes classes. Nous avons mis en œuvre les opérations CRUD pour garantir une gestion efficace des données, avec une version en fichier texte et une autre en SQLite3.

Les tests effectués ont permis de valider le bon fonctionnement des différentes fonctionnalités, en particulier l'ajout, la modification, la suppression et l'affichage des

données. Ils ont aussi mis en évidence certaines limitations, notamment liées aux performances et à la manipulation des fichiers textes, confirmant ainsi l'intérêt de l'utilisation d'une base de données pour un projet évolutif.

V. Limites et Perspectives d'Amélioration

V.1 Limites du projet

Malgré sa fonctionnalité de base, l'application présente certaines limites :

- **Stockage en fichier texte rudimentaire** : L'utilisation d'un fichier texte comme base de stockage rend la gestion des données moins efficace. Il n'y a pas de structure relationnelle et la manipulation des informations devient plus complexe à mesure que le volume de données augmente.
- **Absence d'interface graphique** : L'application fonctionne uniquement en mode console, ce qui peut rendre son utilisation moins intuitive pour des utilisateurs non techniques.
- **Gestion limitée des erreurs** : Bien que des contrôles de base soient effectués, l'application pourrait être améliorée pour mieux gérer les erreurs et les incohérences dans les saisies utilisateurs.
- **Pas de gestion des droits utilisateurs** : Toute personne exécutant le script peut modifier ou supprimer les données sans restriction, ce qui pourrait poser des problèmes de sécurité dans un usage réel.
- **Performance limitée avec SQLite3** : Bien que SQLite3 soit une solution légère et efficace pour des bases de données locales, elle n'est pas optimisée pour gérer un grand volume de données simultanément ou pour un usage en environnement multi-utilisateur.

V.2 Perspectives d'amélioration

Plusieurs améliorations pourraient être envisagées pour rendre le projet plus robuste et plus performant :

- **Migration vers un format de stockage plus structuré** : Remplacer le fichier texte par un fichier JSON ou un format CSV permettrait une meilleure structuration des données tout en restant simple à manipuler.
- **Développement d'une interface utilisateur** : Une interface graphique développée avec **Tkinter** ou **PyQt** améliorerait l'expérience utilisateur et rendrait l'application plus accessible.

- **Ajout d'un système d'authentification** : Implémenter un système de connexion avec des rôles (admin, utilisateur) permettrait de sécuriser les accès et de mieux contrôler les actions sur les données.
- **Optimisation des performances avec une base de données plus avancée** : Une migration vers **MySQL** ou **PostgreSQL** permettrait de mieux gérer de grandes quantités de données et d'améliorer la scalabilité du projet.
- **Ajout de nouvelles fonctionnalités** : Comme l'exportation des données au format PDF ou Excel, la gestion avancée des stocks avec des alertes en cas de rupture, ou encore des statistiques sur les ventes et les commandes.
- **Amélioration de la gestion des erreurs** : Intégrer un meilleur système de gestion des exceptions pour éviter les interruptions du programme et améliorer la robustesse du système.

V.3 Conclusion

Bien que l'application remplisse les objectifs initiaux en assurant la gestion des entités définies, certaines limitations ont été relevées. L'utilisation d'un **fichier texte** pour le stockage présente des contraintes en matière de recherche et de modification des données, tandis que la version **SQLite3**, bien plus performante, reste limitée à une utilisation locale sans gestion d'accès concurrent.

Pour améliorer l'application, plusieurs pistes sont envisageables, comme **l'intégration d'une interface graphique (GUI)** pour rendre l'outil plus convivial, l'implémentation d'un **système d'authentification** pour mieux gérer les utilisateurs, ou encore la migration vers un **SGBD plus avancé** pour supporter un usage à plus grande échelle.

Conclusion Générale

Ce projet de gestion en Python a permis de mettre en œuvre les concepts fondamentaux de la programmation orientée objet et de la gestion des données en adoptant deux approches de stockage différentes. Il a offert une opportunité d'expérimenter les opérations CRUD sur des entités courantes, tout en respectant des contraintes spécifiques.

L'application développée, bien que fonctionnelle, présente encore des limitations, notamment en termes d'ergonomie, de sécurité et de performance. Des améliorations peuvent être envisagées pour rendre le système plus efficace, notamment en intégrant une interface graphique, un système d'authentification et un stockage plus performant.

En conclusion, ce projet a constitué une bonne introduction à la gestion des données en Python et a permis d'explorer les avantages et inconvénients de différentes méthodes de stockage. Il pourrait servir de base pour des développements plus avancés dans le domaine des applications de gestion.